

Python Grundlagen

Bisher haben wir lediglich mit den Befehlen aus dem `turtle`-Modul gearbeitet, um eine kleine Schildkröte zu steuern und Figuren zu generieren. Python kann jedoch viel mehr als nur dieses grafische Modul. Wie jede Programmiersprache folgt auch Python bestimmten Regeln und Muster, damit wir sinnvoll mit dem Computer arbeiten können. Diese Elemente werden wir uns genauer anschauen.

Grundelemente einer Programmiersprache

1. **Variablen und Zuweisungen**
Der Computer muss sich Sachen merken können.
1. **Ein- und Ausgabe**
Der Computer muss Input entgegennehmen können und Output ausgeben können.
1. **Datentypen**
Der Computer hat Regeln, was er mit welchen Elementen machen darf.
1. **Operatoren**
Der Computer muss wissen, was er mit den verschiedenen Elementen machen kann (rechnen, vergleichen, ...).
1. **Kontrollstrukturen**
Der Computer muss Entscheidungen treffen und Dinge wiederholen können.
1. **Funktionen**
Der Computer muss Aufgaben bündeln und wiederverwenden können.

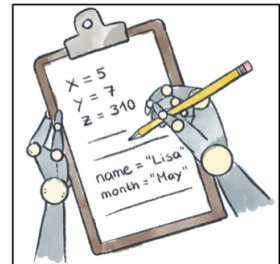
1. Variablen und Zuweisungen

Damit der Computer ein Programm ausführen kann, muss er sich Sachen merken können. Die gespeicherten Werte nennt man **Variablen**.

Variablen und Zuweisungen

Eine **Variable** benutzt man, um einen Wert zu speichern (wie ein Platzhalter).

Sie wird mit einer **Zuweisung** (=) erstellt, wobei links der Name und rechts der Wert der Variable steht.



Zuweisungen



```
1 x = 5
2 monat = "Januar"
3 summe = 1 + 256 + 42
```

Die **Werte der Variablen** können Zahlen, Text oder auch Berechnungen sein.

In der ersten Zeile erhält die Variable `x` den Wert `5`. In der zweiten Zeile bekommt die Variable `monat` den Wert `"Januar"`. Dies ist ein Text, der immer in Anführungszeichen stehen muss, damit der Computer ihn als solchen versteht.

Den **Namen der Variablen** darf man (fast) frei wählen:

- erlaubt sind Buchstaben (klein und gross), Zahlen und der Unterstrich `_`
- der Name darf nicht mit einer Zahl beginnen

Im Verlauf des Programms können sich die Werte von Variablen ändern (man kann sie also überschreiben).

Variablen überschreiben



```
1 x = 5
2 x = 7 # der Wert von x ist neu 7
```

Mit dem Symbol `#` können wir Code in Python **kommentieren**. Alles, was nach dem `#` auf derselben Zeile folgt, wird vom Computer ignoriert. Damit können wir uns Notizen machen.

Aufgabe 1

Gegeben ist eine Liste von möglichen **Variablennamen**. Welche sind laut den Regeln für Python gültig?

```
Summe_1 = 433      _my_name = "Pikachu"      summe1 = 63      *myname = "Glurak"
8Bit = 256          Byte8NumB3r = 16          my name = "Flegmon"      L337Code = 42
```

Wie kann man schnell überprüfen, ob ein Variablenname okay ist oder nicht?

2. Ein- und Ausgabe

Damit wir mit dem Computer arbeiten können, braucht eine Programmiersprache zwei Dinge: Wir müssen **Eingaben (Input)** machen können, und der Computer muss diese verarbeiten und uns danach **Resultate (Output)** zurückgeben.



Ein- und Ausgabe

In Python gibt es zwei Funktionen (Befehle) dafür.

- `input()` Damit fragt der Computer nach einem Input vom Benutzer.
- `print()` Damit können Sie beliebige Sachen in die Kommandozeile (Ausgabefenster) ausdrucken lassen.

Benutzer-Eingabe und Resultat-Ausgabe



```
1 erste_zahl = input("Erste Zahl:")
2 zweite_zahl = input("Zweite Zahl:")
3 print(erste_zahl + zweite_zahl)
```

Um Programme wirklich nützlich machen zu können, braucht es meistens **Inputs** vom Benutzer und ein Resultat als **Output**, das für den Benutzer hilfreich ist.

Beispiele print() und input()

print()

Testen Sie mal folgende Zeilen, indem Sie sie in Ihrem Thonny laufen lassen. Stimmt das Ergebnis mit Ihren Erwartungen überein?

```
1 print("2+3")
```

```
1 print(2+3)
```

Wenn Sie Text und Zahlen gleichzeitig drucken möchten, müssen Sie es mit einem Komma trennen:

```
1 print("Die Summe ist: ", 5)
```

input()

Testen Sie auch den neuen Befehl `input()`. Was passiert bei der folgenden Zeile?

```
1 input("Wie heisst du? ")
```

Wenn man möchte, dass der Benutzer als Input nur eine ganze Zahl eingeben darf, müssen wir noch folgende Anpassung vornehmen:

```
1 int(input("Wie alt bist du?"))
```

Falls der Benutzer keine Zahl eingibt, kommt ein Fehler.

Jetzt können wir auch alles miteinander verbinden, was wir bis jetzt gelernt haben:

```
1 name = input("Wie heisst du? ")
2 print("hallo"*2,name)
```

Aufgabe 2

Erstellen Sie eine Variable mit dem Namen `my_age`. Weisen Sie ihr den Wert Ihres Alters zu. Damit Sie sehen, ob es funktioniert hat, benutzen Sie in der nächsten Zeile `print(my_age)`, um die Variable auszugeben.

Aufgabe 3

Können Sie das Programm so anpassen, dass der Computer Sie mit `input()` fragt, wie alt Sie sind und er dann die Zahl wieder ausgibt? Die Kommandozeile sollte so aussehen:

```
>>> %Run -c $EDITOR_CONTENT
Wie alt sind Sie?29
Ihr Alter ist: 29
```

3. Datentypen

Wenn der Computer mit Variablen zu tun hat, muss er wissen, zu was für einem Typ sie gehören. Wenn er weiss, dass es sich um einen Text handelt, dann weiss er auch sofort, dass nicht alle Operatoren funktionieren werden.

Name	Abkürzung	Beschreibung
Integer	<code>int</code>	Das ist der Datentyp für ganze Zahlen . Alle Zahlen in einem Programm, die keine Dezimalstellen haben, sind ganze Zahlen.
Float	<code>float</code>	Floats sind Zahlen mit Dezimalstellen , welche auch Gleitkommazahlen genannt werden. Jede Zahl, die Dezimalstellen in einem Programm hat, gehört zu diesem Datentyp.
String	<code>str</code>	Strings sind Ketten von beliebigen Zeichen. Einfacher gesagt: Text . Alles, was in einem Programm in Anführungszeichen geschrieben steht, ist vom Datentyp String.
Boolean	<code>bool</code>	Boolean sind Wahrheitswerte . Davon gibt es nur zwei: True und False .

Datentypen

Variablen gehören in Python jeweils einem **Datentyp** an. Dieser bestimmt dann, was man alles mit der Variable tun kann. Man kann zum Beispiel bei einem String nicht dividieren.

Wenn Sie eine Variable `x` haben, können wir mit dem Befehl `print(type(x))` den Datentyp herausfinden.

Datentyp herausfinden

```
1 alter = 17
2 print(type(alter))
3 name = "harry"
4 print(type(name))
```

Bestimmter Datentyp als Input

```
1 alter = int(input("Ganze Zahl eingeben:"))
2 groesse = float(input("Zahl eingeben:"))
3 name = str(input("Bitte Text eingeben:"))
```

Nicht alle Datentypen sind miteinander kompatibel. Zum Beispiel kann Python einen Integer und einen String nicht miteinander addieren. Einen Integer mit einem Float hingegen schon. Darum wollen wir bei den Inputs meistens nur ganz bestimmte Datentypen zulassen.

Manchmal möchte man Datentypen «ineinander umwandeln». Z.B. wenn man in einer `print`-Ausgabe Text und Zahlen in einem Satz darstellen möchte. Zwei Texte lassen sich mit einem `+` aneinanderhängen (siehe Zeile 1). Mit Text und Zahl funktioniert dies aber nicht (siehe Zeile 3), denn dies führt zu einem Fehler. Man kann die Zahl `x` jedoch mit `str(x)` in einen Text umwandeln um mit `print` auszugeben (Zeile 4) oder Text und Zahl (ohne Umwandlung) mit einem Komma trennen (Zeile 5).

Text und Zahlen im Output kombinieren

```
1 print("hallo" + " wie gehts?")
2 x = 5
3 print("Das Resultat lautet " + x)
4 print("Das Resultat lautet " + str(x))
5 print("Das Resultat lautet ", x)
```

Aufgabe 4

Bis jetzt konnte man bei Ihrem Code aus Aufgabe 3 auch Texte oder Kommazahlen eingeben. Passen Sie Ihren Code so an, dass nur noch ganze Zahlen als Input erlaubt sind.

(Das heisst, wenn der Benutzer etwas anderes als eine ganze Zahl eingibt, soll ein Fehler erscheinen.)

Aufgabe 5

Gegeben sind verschiedene Variablen. Bestimmen Sie zuerst ohne Computer, welchem Datentyp sie jeweils zugehören und überprüfen Sie Ihre Lösung dann mit Thonny.

<code>"2+3"</code>	<code>2+3</code>	<code>2.0+3</code>	<code>2.0+3.0</code>
<code>"2"*3</code>	<code>"2"+"2"</code>	<code>"True"</code>	<code>True</code>

Aufgabe 6

Welche der Ausdrücke sind gültig in Python?

<code>3**2+4**2</code>	<code>("go"*3)+"yes"</code>	<code>3.14*10*10</code>
<code>"go"+3</code>	<code>"go"*3</code>	<code>3.14+10</code>

Aufgabe 7

Erklären Sie in Ihren eigenen Worten, warum `print("15+10")` nicht das gleiche gibt wie `print(15+10)`.

4. Operatoren

Lösungen

Lösung 1

gültig:

```
Summe_1 = 433
summe_1 = 123
_my_name = "Pikachu"
L337Code = 42
summe1 = 63
Byte8Numb3r = 16
```

ungültig:

```
*myname = "Glurak"
8Bit = 256
my name = "Flegmon"
```

Variablenname auf Gültigkeit überprüfen:

Zuweisung mit dem Variablennamen in Thonny eingeben und den Code laufen lassen. Bei einem ungültigen Namen wird eine Fehlermeldung erscheinen.

Lösung 2

```
1 my_age = 17
2 print(my_age)
```



Lösung 3

```
1 my_age = input("Dein Alter?")
2 print("Ihr Alter ist", my_age)
```



Lösung 4

```
1 my_age = int(input("Dein Alter?"))
2 print("Ihr Alter ist", my_age)
```



Lösung 5

"2+3" → str	2+3 → int
2.0+3 → float	2.0+3.0 → float
"2"*3 → str	"2"+"2" → str
"True" → str	True → bool

Lösung 6

Nur "go"+3 ist ungültig. Alle anderen sind gültig.

Lösung 7

Alles in Anführungszeichen wird als Text behandelt. Deswegen ist für Python der Ausdruck "15+10" reiner Text und wird nicht berechnet. Der Ausdruck `print("15+10")` liefert den Output 15+10 (als Text) und der Ausdruck `print(15+10)` liefert 25 (als Zahl).